COS30018 – Intelligent Systems

Task C.5 – Machine Learning 2

This is what I have implemented so far for task C.5:

I created a new parameter in the “build_model” function called “output_size” and I set it to be 1 as a default value, I also change the “Dense” of the output layer to take the “output_size” because it will be useful in the later part to predict a “k” number of days.

And then I created a new function called “create_multistep_sequences”:

```
def create_multistep_sequences(scaled_df, prediction_days, future_steps):
    X, y = [], []
    data_arr  = scaled_df.values
    close_idx = list(scaled_df.columns).index("Close")
    for i in range(prediction_days, len(data_arr) - future_steps + 1):
        X.append(data_arr[i - prediction_days:i, close_idx])
        y.append(data_arr[i:i + future_steps, close_idx])
    X = np.array(X)
    X = X.reshape(X.shape[0], X.shape[1], 1)
    return np.array(X), np.array(y)
```

Basically, the “prediction_days” are the previous days used as input where the “future_steps” are the ouput for the next “k” days. For example, if the “Close” feature data are [1,2, …, 9], and the “prediction_days” and “future_steps” was set to 3 and 2, then if the input X is [1,2,3], then the output y is [4,5]. This function will be handful later when I used it to produce a sequence of output.

The next thing I did was to create this function to predict multiple days in the future:

```
def predict_multistep(result, future_steps=5, prediction_days=60,
                      layer_type="LSTM", layer_sizes=[50, 50],
                      epochs=25, batch_size=32):
    close_scaler = result["scalers"]["Close"]
    train_df = result["train_df"].copy().astype(float)
    test_df  = result["test_df"].copy().astype(float)
    for col in result["feature_columns"]:
        scaler = result["scalers"][col]
        train_df[col] = scaler.transform(train_df[col].values.reshape(-1, 1)).flatten()
        test_df[col]  = scaler.transform(test_df[col].values.reshape(-1, 1)).flatten()

    X_train, y_train = create_multistep_sequences(train_df, prediction_days, future_steps)
    X_test,  y_test  = create_multistep_sequences(test_df,  prediction_days, future_steps)

    model = build_model(
        input_shape  = (prediction_days, 1),
        layer_type   = layer_type,
        layer_sizes  = layer_sizes,
        output_size  = future_steps,
    )
```

```python
model.fit(X_train, y_train, epochs=epochs, batch_size=batch_size, verbose=1)

predicted_scaled = model.predict(X_test)

predicted_prices = close_scaler.inverse_transform(
    predicted_scaled.reshape(-1, 1)
).reshape(predicted_scaled.shape)

actual_prices = close_scaler.inverse_transform(
    y_test.reshape(-1, 1)
).reshape(y_test.shape)

plt.figure(figsize=(14, 5))
plt.plot(actual_prices.flatten(),    color="black", label="Actual")
plt.plot(predicted_prices.flatten(), color="green", label=f"Predicted ({future_steps}-step)")
plt.title(f"Multistep Prediction — {future_steps} days ahead")
plt.xlabel("Time steps")
plt.ylabel("Close Price (AUD)")
plt.legend()
plt.tight_layout()
plt.show()

last_window = X_test[-1:]
next_k = model.predict(last_window)
next_k = close_scaler.inverse_transform(next_k.reshape(-1, 1)).flatten()
print(f"\nMultistep — next {future_steps} days prediction: {np.round(next_k, 4)}")

return model
```

This function takes in multiple parameters and then it is divided into 7 stages:

- The first stage is to scale the data.
- The second stage is to transform the X and y into sequences.
- The third stage is to build and train the model itself using the pre-implemented “build_model” function, but this time, I set the “output_size” to take the value of “future_steps”.
- The fourth stage is to predict using the test data.
- The fifth stage is to convert the result of the test prediction back to actual numbers.
- The sixth stage is to plot the whole data into a figure to compare the test prediction with the original data.
- The final stage will be actually predicting the stock price using the last 60 days as input and then predict the next 5 days.

The next thing I did was to create a new function called “predict_multivariate”’

```python
#-----------------------------------------------------------------------------
# Multivariate prediction
#-----------------------------------------------------------------------------
def predict_multivariate(result, target_day=1, prediction_days=60,
                         layer_type="LSTM", layer_sizes=[50, 50],
                         epochs=25, batch_size=32):
    close_scaler = result["scalers"]["Close"]
    n_features   = result["X_train"].shape[2]
    train_df = result["train_df"].copy().astype(float)
    test_df  = result["test_df"].copy().astype(float)

    for col in result["feature_columns"]:
        scaler = result["scalers"][col]
        train_df[col] = scaler.transform(train_df[col].values.reshape(-1, 1)).flatten()
        test_df[col]  = scaler.transform(test_df[col].values.reshape(-1, 1)).flatten()

    def create_multivariate_sequences(scaled_df):
            X, y = [], []
            data_arr  = scaled_df.values
            close_idx = list(scaled_df.columns).index("Close")
            for i in range(prediction_days, len(data_arr) - target_day + 1):
                X.append(data_arr[i - prediction_days:i])
                y.append(data_arr[i + target_day - 1, close_idx])
            return np.array(X), np.array(y)

    X_train, y_train = create_multivariate_sequences(train_df)
    X_test,  y_test  = create_multivariate_sequences(test_df)

    model = build_model(
        input_shape  = (prediction_days, n_features),
        layer_type   = layer_type,
        layer_sizes  = layer_sizes,
        output_size  = 1,
    )
```

Basically, it is almost the same thing with the previous function, except several thing that are in this one: the code to generate the y array sequence is “y.append(data_arr[i + target_day - 1, close_idx])” which using a new parameter called “target_day” instead of “future_steps”, which basically set the y array sequence to only contain 1 single value of the specific wanted day in the future, and the “output_size” of the model is also set to 1, also the input of this function can be from multiple sources of features (right now I only set it to take from “Close” feature).

The main differences between these 2 functions are that the “predict_multistep” function predict a sequence of multiple days in the future but “predict_multivariate” function only predict a specific set day in the future. The “predict_multivariate” can be more flexible than the “predict_multistep” if the point of stock price you want to predict is far from the lasted day of the dataset. And also, the “predict_multistep” function only take input from “Close” feature where the “predict_multivariate” function can take input from multiple features.

Finally, I created a new function called “predict_multivariate_multistep” which is a combination of those 2 previous functions.

```python
def predict_multivariate_multistep(result, future_steps=5, prediction_days=60,
                                   layer_type="LSTM", layer_sizes=[50, 50],
                                   epochs=25, batch_size=32):
    close_scaler = result["scalers"]["Close"]
    n_features   = result["X_train"].shape[2]
    train_df = result["train_df"].copy().astype(float)
    test_df  = result["test_df"].copy().astype(float)

    for col in result["feature_columns"]:
        scaler = result["scalers"][col]
        train_df[col] = scaler.transform(train_df[col].values.reshape(-1, 1)).flatten()
        test_df[col]  = scaler.transform(test_df[col].values.reshape(-1, 1)).flatten()

    def create_combined_sequences(scaled_df):
        X, y = [], []
        data_arr  = scaled_df.values
        close_idx = list(scaled_df.columns).index("Close")
        for i in range(prediction_days, len(data_arr) - future_steps + 1):
            X.append(data_arr[i - prediction_days:i])
            y.append(data_arr[i:i + future_steps, close_idx])
        return np.array(X), np.array(y)

    X_train, y_train = create_combined_sequences(train_df)
    X_test,  y_test  = create_combined_sequences(test_df)

    model = build_model(
        input_shape  = (prediction_days, n_features),
        layer_type   = layer_type,
        layer_sizes  = layer_sizes,
        output_size  = future_steps,
    )
```

It can take inputs from multiple source and it can predict a set “k” days stock price in the future, right now I only set it to take input from “Close” feature only but you can change it if you want.

At the end of the program, I set up a configuration to test these 3 new functions out:

```python
print("\n" + "="*60)
print("Multistep Prediction (5 days ahead)")
print("="*60)
predict_multistep(
    result           = result,
    future_steps     = 5,
    prediction_days  = PREDICTION_DAYS,
    layer_type       = "LSTM",
    layer_sizes      = [50, 50],
    epochs           = 25,
    batch_size       = 32,
)

print("\n" + "="*60)
print("Multivariate Prediction (day +3)")
print("="*60)
predict_multivariate(
    result           = result,
    target_day       = 3,
    prediction_days  = PREDICTION_DAYS,
    layer_type       = "LSTM",
    layer_sizes      = [50, 50],
    epochs           = 25,
    batch_size       = 32,
)

print("\n" + "="*60)
print("Multivariate + Multistep (5 days ahead)")
print("="*60)
predict_multivariate_multistep(
    result           = result,
    future_steps     = 5,
    prediction_days  = PREDICTION_DAYS,
    layer_type       = "LSTM",
    layer_sizes      = [50, 50],
    epochs           = 25,
    batch_size       = 32,
)
```